PointShuffler: Accelerating Point Cloud Neural Networks on General-Purpose GPUs

Yangfan Li*
School of Computer Science and Engineering, Central South University

Zhengjie Jin*
School of Computer Science and Engineering, Central South University

Yue Tian
School of Computer Science and Engineering, Central South University

Mengquan Li
College of Integrated Circuits, Hunan University

Fengxiao Tang†
School of Computer Science and Engineering, Central South University

Ming Zhao
School of Computer Science and Engineering, Central South University

Cen Chen†
School of Future Technology, South China University of Technology
Pazhou Laboratory

Abstract

Point Cloud Neural Networks (PCNNs) have emerged as a vital tool for latency-sensitive 3D perception applications, such as autonomous driving and AR/VR. However, their inherent computational redundancy—arising from excessive global sampling/search operations and repeated feature updates/aggregations caused by shared neighbors—severely constrains execution efficiency. More critically, conventional redundancy elimination methods usually introduce operations that are highly GPU-unfriendly, resulting in high memory overhead, increased branch divergence, irregular memory access, and serial dependencies, which together pose a significant challenge to PCNN acceleration.

To this end, we propose PointShuffler, the first-of-its-kind GPU-friendly inference engine that efficiently eliminates PCNN redundancy with preserved accuracy. It consists of three enabling optimizations: (1) Network Architecture Transformation, which can convert a typical PCNN into an architecture variant with well-eliminated redundancy while negligible accuracy losses; (2) Data Orchestration, which arranges the irregular data access inherent in 3D point clouds into deterministic and contiguous data access, to fundamentally alleviate the costs from data access, branch divergence, and data movement; and (3) Operation Parallelization, which enables parallel sampling and neighbor searching by eliminating serial dependencies lie in the core operations of PCNNs. Extensive experimental results demonstrate the effectiveness of PointShuffler, achieving an average of 6.15× speedup and at least 5.30× memory saving without accuracy loss. The codes of PointShuffler are released at https://github.com/Jeff5trick/PointShuffler.

CCS Concepts: • **Computing methodologies** → **Parallel computing methodologies**; **Computer vision**.

Keywords: Point Cloud Neural Networks, GPU Acceleration, Redundancy Elimination

ACM Reference Format:
Yangfan Li, Zhengjie Jin, Yue Tian, Mengquan Li, Fengxiao Tang, Ming Zhao, and Cen Chen. 2026. PointShuffler: Accelerating Point Cloud Neural Networks on General-Purpose GPUs. In *European Conference on Computer Systems (EUROSYS '26), April 27–30, 2026, Edinburgh, Scotland Uk.* ACM, New York, NY, USA, 16 pages. https://doi.org/10.1145/3767295.3803611

1 Introduction

As a fundamental 3D data representation, point clouds capture geometric reality through massive, irregularly distributed points. Advanced sensing technologies like LiDAR have driven their adoption in latency-critical applications, including autonomous driving and AR/VR systems, where point cloud neural networks (PCNNs) [6, 25, 26, 35] serve as the computational backbone in real-time 3D perception models [18, 29, 37]. Modern PCNNs typically employ a four-stage computational architecture, including global sampling, global neighbor search, feature update, and aggregation. However, the intrinsic redundancy in this architecture severely constrains their computational efficiency [3, 4, 9, 17, 43]. Specifically, there are (i) excessive computations from repeatedly sampling and searching the entire space, and (ii) repeated feature updates and aggregations caused by

*These authors contributed equally to this work.
†Corresponding Authors.

EUROSYS '26, Edinburgh, Scotland Uk

ACM ISBN 979-8-4007-2212-7/26/04
https://doi.org/10.1145/3767295.3803611

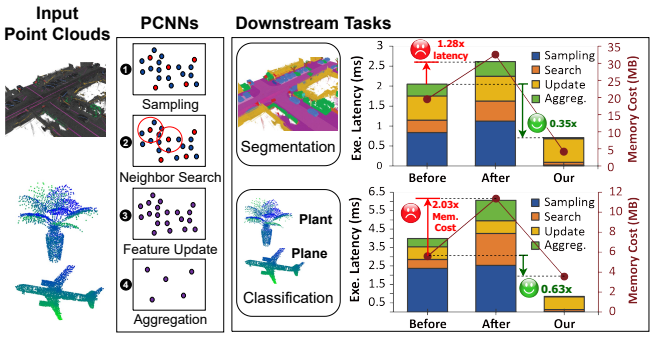

Figure 1. Performance degradation (latency & memory cost) when applying existing PCNN redundancy optimization techniques to GPUs.

shared neighbor points. Specifically, for a typical classification PCNN model, existing research demonstrated that approximately 98% of global sampling, 77% of global neighbor search, 94% of feature update, and 75% of *aggregation* operations are unnecessary [3, 9, 17]. Processing large-scale scenes with millions of points therefore struggles to satisfy the sub-100 ms latency demanded by time-sensitive tasks without compromising accuracy.

Early work Mesorasi [9] first revealed the massive redundancy in the feature-update stage and introduced a delayed aggregation scheme that removes it, achieving solid speedups on both GPUs and a prototype ASIC. Nevertheless, because it leaves the redundancies in sampling, neighbor search, and aggregation untouched, the overall pipeline remains far from optimal. Later studies for redundancy optimization moved to ASIC designs and tackled redundancies in other stages: GDPCA[3] uses a fixed-size cubic partition to curb aggregation and neighbor search cost; SimDiff[17] and ParallelNN[4] adopt octree-based partitions to accelerate neighbor search and sampling. However, the practical impact of these ASIC-centric solutions is fundamentally limited, as their customized hardware has high deployment costs and is inflexible to rapid algorithmic evolution. In contrast, general-purpose GPUs (such as NVIDIA Drive Orin AI platform for autonomous vehicles) remain the de-facto deployment platform for 3D perception thanks to their mature CUDA software stack and low deployment costs.

Unfortunately, these redundancy optimization techniques do not translate well to GPUs. Worse still, directly applying these techniques to GPUs can even degrade performance, as our experiments demonstrate (Fig. 1). This is because they introduce excessive irregularity in memory access, layout patterns, and computational operations, which directly contradicts GPU architectures optimized for regular, parallelizable workloads. Specifically, ① Spatial partitioning of irregular point distributions leads to severe memory padding overhead under the GPUs' fixed-size memory allocation model. ② Computation-reuse schemes introduce control-flow flags that trigger branch divergence and irregular access of the intermediate result, clashing with the Single-Instruction Multiple-Thread (SIMT) execution model. ③ Core operations such as farthest-point sampling (FPS) and shared-neighbor identification exhibit serial dependencies that significantly harm parallelism.

To this end, we propose **PointShuffler**, the first end-to-end inference engine that achieves efficient PCNN acceleration by eliminating computational redundancies on general-purpose GPUs, while preserving task accuracy. PointShuffler can transform a given PCNN into a GPU-friendly variant with three core optimization techniques. **Network Architecture Transformation**: We propose an *adaptive search region* to avoid unnecessary global sampling & neighbor search operations with high search precision, and design a *compute-once-reuse* strategy that eliminates repeated feature updates and aggregations. **Data Orchestration**: We build a *hierarchical spatial index* (HSI) that reshapes irregular point distributions into deterministic, contiguous memory segments without any padding, enabling data movement-free and coalesced access and branch-free aggregation. **Operation Parallelisation**: Building upon the HSI, we effectively parallelize core PCNN operations. These include: the *parallel strided sampling* to replace sequential FPS; the *lock-free shared-neighbor identification* to avoid costly synchronizations and minimize the shared neighbor search overhead; and the *branch-free segmented aggregation* that fully aligns with the GPU's SIMT execution model when reusing the dynamically computed intermediate results. Implemented in CUDA C/C++ and seamlessly integrated with PyTorch, PointShuffler requires no model re-training, offers user-friendly APIs, and is released as open source. Comprehensive evaluations on standard benchmarks demonstrate an average of **6.15**× speedup and at least **5.30**× memory savings over state-of-the-art GPU baselines, without degrading accuracy.

2 Background and Analysis

2.1 Point Cloud Neural Network

A point cloud is an unordered set of massive dispersed points distributed in 3D space, formally represented as $X = \{x_1, x_2, ..., x_n\}$. Each point $x_i \in R^F$ is uniquely defined by an F-dimensional feature vector and spatial coordinates (x, y, z). Unlike RGB images with uniform grid structures, point clouds exhibit inherent characteristics of sparsity and geometric sensitivity, because they are only distributed in regions where objects exist or on their surface. These features necessitate specialized neural architectures for effective point cloud processing.

Thanks to high accuracy, point cloud neural networks (PCNNs) [6, 25, 26, 35] have been widely explored for directly processing unordered 3D point clouds while preserving raw geometric fidelity (i.e., without voxelization). PCNNs typically comprise a stack of point-based Set Abstraction (SA) layers plus a downstream task head (e.g., classification/segmentation); each SA layer follows the pipeline below[25]:

① **Global Sampling.** Given an input point cloud X, this stage selects m representative points from the whole space through downsampling, as formulated:

$$X' = \mathcal{S}(X) = \{x'_1, x'_2, \ldots, x'_m\} \in \mathbb{R}^{m \times F}, \tag{1}$$

where $\mathcal{S}(\cdot)$ typically uses Farthest Point Sampling (FPS) to maximize spatial coverage.

② **Global Neighbor Search.** For every sampled point $x'_j \in X'$, this stage searches the entire space X and identifies all the spatially adjacent neighbors for it, usually through K-nearest neighbor (KNN) or ball query[23, 25].

③ **Feature Update:** After obtaining all the neighbors of a sampled point, for each neighbor pair $(x'_j, x_k) \in X' \times N(x'_j)$, its edge features are updated via a linear transformation:

$$e'_{jk} = \sigma\left(w \cdot \left(x_k - x'_j\right)\right), \tag{2}$$

where e'_{jk} is the updated edge feature, w is a weight matrix, and σ is an activation function (e.g., ReLU). This stage expands the feature dimension from $n \times F$ to $m \times k \times F$.

④ **Aggregation:** Finally, each sampled point collects the information of its neighbors by edge feature aggregation:

$$\hat{x}'_j = \underset{1 \le k \le K}{\mathcal{A}} \left(e'_{jk}\right). \tag{3}$$

where $\mathcal{A}(\cdot)$ usually implements max/average pooling.

2.2 Computational Redundancy Analysis and Optimization of PCNNs

Despite their success, PCNNs exhibit inherent computational redundancies that stem from their architecture and core operations, mainly including:

Excessive global Sampling and Neighbor Search. Global sampling and searching of the entire point space incurs significant but often unnecessary computational overhead. Specifically, during the sampling stage, the computational complexity of FPS grows quadratically with the number of points, making pairwise distance calculations over the entire massive-scale point cloud prohibitively expensive. Similarly, for neighbor search, performing a global search is also highly inefficient. This is because points in a point cloud typically cluster on object surfaces, meaning their neighbors are usually spatially close, rendering a global scan unnecessary. This claim is supported by our experimental results in Fig. 2, which shows the relationship between search range and accuracy. For instance, on the ModelNet40 dataset, searching within a local region that covers only 0.1% of the total space can achieve 98% of the search accuracy obtained by searching the entire space.

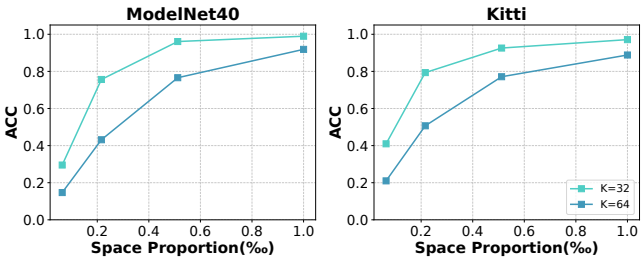

Figure 2. The relation of search space proportion and accuracy of neighbor retrieval (relative to a full global search).

Repeated Update and Aggregation Operations Due to Shared Neighbors. After obtaining the neighbors of every sampled target point, some of the neighbor points are shared by multiple target points (dubbed *Shared Neighbors*), while the remaining neighbors are the set of the *Unique Neighbors* of specific target points. As PCNNs perform feature update and aggregation for each neighbor pair, the features of shared neighbors are repeatedly calculated. More importantly, statistical analysis demonstrated that shared neighbors would account for up to 75.2% of the entire neighbor composition [3]. Taking Fig. 3 as an example, there are two adjacent target points P_1 and P_2. Their neighbors can be divided into three sets, namely two unique neighbor sets (i.e., U_1 and U_2) and one shared neighbor set (i.e., S). When updating and aggregating the features of P_1 and P_2, each neighbor point in S is calculated twice. It will get worse for large dense point clouds.

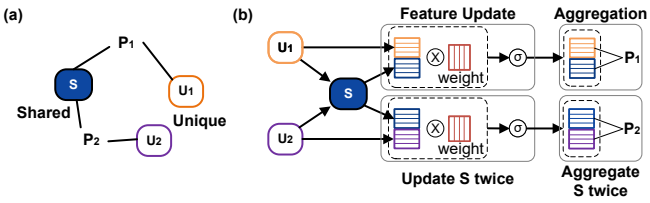

Figure 3. Repeated feature update and aggregation operations due to shared neighbors.

Limitations of Existing PCNN redundancy optimization. Recent work has introduced new methods to optimize PCNNs by reducing redundancy. GDPCA [3] introduces an ASIC design which employed a fixed-size cubic region partitioning scheme to effectively reduce overhead for aggregation and neighbor searching, as well as the bit-level feature update redundancy. Mesorasi [9] proposed delay aggregation to reduce the number of feature updates and demonstrated its effectiveness on both ASIC and GPU. ParallNN [4] proposed an octree-based partitioning scheme to shrink the

search space and accelerate the neighbor search stage with a parallel ASIC design.

Despite these theoretical performance advantages, the practical adoption of these existing methods faces significant constraints. This limited adoption stems from two primary factors: (1) Most of them are ASIC-centric designs and validated via simulation. Specialized hardware solutions currently lack the flexibility to accommodate new or changing algorithms. (2) They often provide partial rather than end-to-end optimizations for the entire PCNN pipeline. For instance, Mesorasi solely targets the feature update stage, ParallelNN focuses on neighbor search, and while GDPCA offers broader coverage, it still omits optimizations for the sampling stage.

In contrast, general-purpose GPUs maintain dominance in practical PCNN deployments due to their proven advantages in flexibility of supporting various algorithms, the mature CUDA ecosystem with optimized libraries and considerable lower development/maintenance costs compared to specific hardware solutions. Therefore, **addressing these redundancies for efficient PCNN acceleration on GPGPUs is one of the crucial prerequisites** for advancing 3D perception applications.

2.3 Challenges in Accelerating PCNNs on GPGPUs via Redundancy Elimination

Mitigating unnecessary computations for efficient PCNN acceleration on GPUs is non-trivial, primarily due to three challenges:

Challenge 1: Padding Memory Explosion under Uneven Point Density. A common way to avoid global sampling and full-space neighbor search is to partition the point cloud into spatial sub-blocks [3]. However, the highly uneven point distributions collide with fixed-capacity GPU kernels. Shown in Fig. 4, because the exact number of points per sub-block is unknown and unpredictable before launching a parallel kernel, implementations need to pre-allocate each sub-block for the worst case, which is the number of points of the entire cloud, leading to severe memory waste on padding. In practice, assuming an n-point cloud is divided into D^3 sub-blocks, every sub-block is allocated an index buffer large enough to hold *all* n points, even if the sub-block is nearly empty. Therefore, the total allocated memory becomes:

$$\text{Total indices} = \underbrace{D^3 \cdot n}_{\text{allocated capacity}} = n + \underbrace{(D^3 - 1) \cdot n}_{\text{extra padding}}.$$

Recent memory management systems, such as PagedAttention [16] and FlashInfer [41], cut KV caches into fixed-size pages and use a page/block table to mitigate memory fragmentation. This fits LLM serving with many concurrent, long-lived sequences, but not our setting. This is because PCNNs require efficiently rebuilding a fresh and short-lived neighbor index at every layer, while paging is designed for append-only and read-heavy indices management workloads to update, look up, and reuse existing entries with CPU-assistance, and is ill-suited to minimizing single-frame latency.

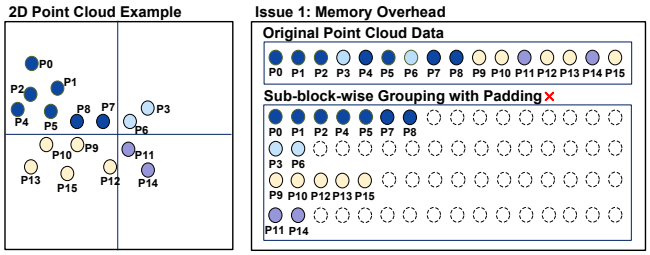

Figure 4. Example of worst-case padding in sampling/search operation. In this case, we divide the point cloud space into 4 sub-blocks. The GPU has to pre-allocate a storage space of 4×16 to save the indices.

Challenge 2: Reuse Induces Control Flow & Memory Irregularity on SIMT GPUs. Computation reuse can, in principle, remove most redundant work in PCNNs. For feature update, the term $w \cdot x_i$ can be computed once and reused for all edges incident to point i (Equ. 9); for aggregation, features of shared neighbors can be computed once and reused across multiple target points (Equ. 9). Theoretical analysis indicates this could eliminate up to 93.75% repeated updates and 75.05% repeated aggregations.

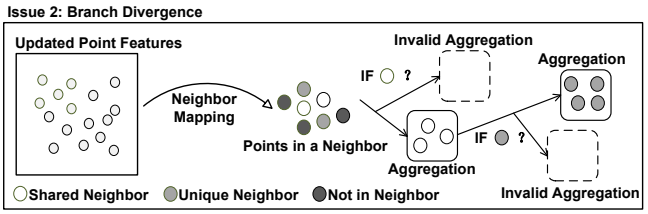

Figure 5. Attempts to reuse intermediate aggregation results usually require 2 control flags, which leads to branch divergence. *Non-neighbors* refer to points in a given sub-block that are not neighbors of the specific target point.

However, translating these theoretical savings into practical GPU speedups is non-trivial, naive computation reuse introduces additional conditional branches (e.g., to judge shared/unique/non-neighbor status) and creates warp/branch divergence that conflicts with the single instruction multiple thread (SIMT) execution of contemporary GPUs. Fig. 5 illustrates this inefficiency. Moreover, enabling reuse leads to memory-access irregularity, as shared-neighbor features are consumed by scattered target threads, yielding indirect, non-contiguous operations.

Challenge 3: Serial Bottlenecks on GPUs from FPS and Shared Neighbor Synchronization. First, FPS performs point-by-point selection with dependencies on prior choices,

which makes it a serial bottleneck for SIMT GPUs. Second, enabling computation reuse during aggregation introduces a synchronization barrier: the shared-neighbor set is only determinable after the neighbor lists for all sampled points in the region (e.g., the sub-block) are available. Because neighbor counts vary across points, progress is gated by the slowest search, causing load imbalance and expensive global synchronization issues. In practice, serialized FPS dominates end-to-end time on large point sets (47.58%–55.72%), and a conventional reuse implementation can even slow the aggregation stage by 2.48× (Fig. 1).

3 The Proposed *PointShuffler* Engine

In this section, we propose the first-of-its-kind inference engine, dubbed PointShuffler, which demonstrates efficient PCNN acceleration on GPGPUs with eliminated computation redundancy.

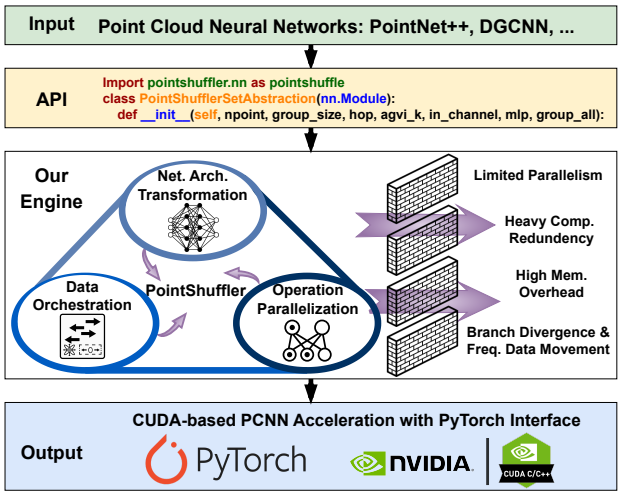

Figure 6. Our PointShuffler inference engine.

3.1 System Overview & Programming Interface

PointShuffler targets on-device deployment on GPU-equipped edge platforms (e.g., AR/VR headsets and autonomous-driving platforms), accelerating PCNN-based point-cloud models for tasks such as classification and segmentation. As shown in Fig. 6, Given a PCNN model as input, our PointShuffler engine transforms the network architecture as well as its execution pipeline (Sec 3.2), shuffles the data access pattern (Sec 3.3) and restructures the serial key operations (e.g., sampling, neighbor search, aggregation) into parallel execution (Sec 3.4). All these 'shuffling' optimizations are tailored for GPUs. As a result, a GPU-friendly PCNN variant is output.

Users define their PCNNs through PointShuffler APIs, a drop-in replacement for standard Set Abstraction operators that adds only two arguments to control the neighbor-search scope and point-cloud partitioning. All the optimizations are implemented in CUDA within these interfaces, and the Python APIs invoke the CUDA back-end via pybind.

The three enabling optimization techniques are summarized as follows:

Enabler-1: Network Architecture Transformation. The transformed PCNN architecture incorporates two key algorithmic advancements to reduce unnecessary operations: First, we propose an adaptive search region approach for sampling and neighbor search to improve the search precision compared to the fixed-size local regions. Second, we propose a shared-neighbor caching strategy to mitigate the repeated computations associated with shared neighbors in adaptive regions.

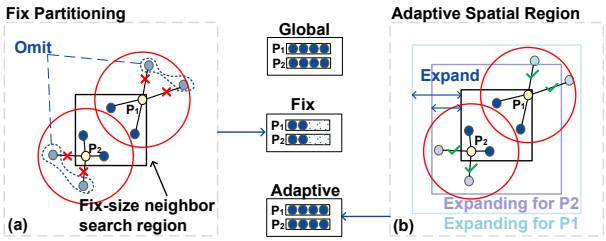

Figure 7. Comparison of using (a) fixed-size search region and (b) adaptive search region (ours). Ours can retrieve neighbors located near partition boundaries, which can be easily ignored (resulting in decreased accuracy).

Enabler-2: Data Orchestration. Although *Enabler-1* removes algorithmic redundancy, realizing these gains on GPUs depends on the data layout to address the padding and conditional flag issues (as detailed in Challenge 1 & 2). We therefore propose a novel unified hierarchical spatial index (HSI), which employs pre-calculated, multi-level offsets to shuffle irregular data properties (like sub-blocks, search region, and unique vs. shared neighbor status) into a deterministic and contiguous data segments layout, requiring no data padding. This structured layout enables GPU kernels to access the necessary indices in a coalesced manner, thereby reducing the need for costly data movement of high-dimensional point features. Furthermore, HSI ensures that points within each segment belong to the same category (e.g., shared or unique neighbors), facilitating intra-segment branch-free aggregation.

Enabler-3: Operation Parallelization. PointShuffler further proposes a set of parallel algorithms, deeply integrated with the HSI data structure, to address the branching and serial dependency issues (Challenge 2 & 3) in core PCNN operations. (1) We propose the parallel strided sampling, which leverages the HSI's spatially coherent memory layout to achieve distance-free, highly parallel sampling while maintaining spatial coverage, effectively replacing sequential FPS. (2) To tackle the synchronization overhead in identifying

shared neighbors, we design a lock-free shared neighbor identification mechanism, which efficiently searches shared neighbors using an inverse identification strategy. (3) We propose the branch-free segmented aggregation, which exploits the HSI's segmented neighbor data to execute computation reuse for shared neighbors in a SIMT-friendly manner.

3.2 Network Architecture Transformation

We convert a typical PCNN into an architecture variant with well-eliminated redundancy and negligible accuracy losses.

Adaptive Search Region. An intuitive approach to eliminating global sampling/search redundancy is to partition the entire point cloud space into a set of D^3 non-overlapping sub-blocks $\{B_d\}_{d=1}^{D^3}$. Operations like sampling and neighbor search are then performed within these sub-blocks. However, existing spatial partitioning schemes, such as GDPCA[3] and ParallelNN[4], often use only the sub-block containing the target point as the search region. As illustrated in Fig. 7(a), when a target point lies near the boundary of its sub-block, this approach may fail to include true neighboring points (found by a global search) that reside in adjacent sub-blocks.

To address the limitation, our adaptive spatial partitioning strategy dynamically defines the search scope for sampling and neighbor search. For each initial sub-block B_d (from a D^3 grid partitioning the entire space), we determine an *adaptive search region*. This region includes B_d itself and expands to neighboring sub-blocks based on an integer hop parameter $h \geq 0$. As illustrated in Fig. 9(b), setting $h = 1$ typically extends the search region to include immediately adjacent sub-blocks (e.g., a 3×3 local region in 2D projections). This ensures comprehensive coverage of true neighbors, especially for points near sub-block boundaries. Let $P(B_d, h)$ denote the set of all points from the input cloud X that fall within the h-hop adaptive search region defined around B_d.

During the sampling stage, instead of sampling from the entire cloud, we sample points X' by applying the sampling function $\mathcal{S}$ within each of these adaptive search regions and take the set union as the final result:

$$X' = \bigcup_{d=1}^{D^3} \mathcal{S}\big(P(B_d, h)\big). \tag{4}$$

For the neighbor search stage, given a target point $x'_j \in X'$, we first identify its associated central sub-block $B(x'_j)$. The search operation $\mathcal{N}$ for its neighbors N_j is then conducted within the adaptive search region $P(B(x'_j), h)$:

$$N_j = \mathcal{N}\left(x'_j, P(B(x'_j), h)\right). \tag{5}$$

Shared Neighbor Caching. To remove redundant work from shared neighbors, we propose a caching scheme to compute each per-point update once and each sub-block shared-neighbor aggregate once, and then reuse them throughout the layer. This design is based on delayed aggregation [3, 9], which defers edge computations until after updates, to reduce operations with negligible impact on accuracy.

First, in the feature-update stage, unlike Eq. 2, which computes an update for every target–neighbor pair, we perform a single per-point update across the cloud and cache it:

$$\tilde{X} = \sigma(w \cdot X) \tag{6}$$

During the aggregation stage, for each sub-block B_d, we first compute and cache the common neighbor set shared by all its target points (the intersection of their neighbor sets):

$$X^d_{\text{shared}} = \bigcap_{j \in B_d} N_j. \tag{7}$$

We then aggregate it once to obtain the sub-block intermediate:

$$\hat{X}_d = \mathcal{A}\Big(\tilde{X}\big[X^d_{\text{shared}}\big]\Big). \tag{8}$$

Finally, the complete aggregation stage is reshaped. For each target point x'_j, the aggregation process by reusing the cached results $\tilde{X}[N_j]$ and $\hat{X}_d$ is as follows:

$$\hat{x'_j} \approx \underset{j \in B_d}{\mathcal{A}} \left(\tilde{X}[N_j],\ \hat{X}_d\right) - \tilde{X}[j]. \tag{9}$$

Transformed Execution Pipeline. As shown in Fig. 8, ① PointShuffler constructs a three-level Hierarchical Spatial Index (HSI), comprising sub-block, search region, and neighbor levels (detailed in Sec. 3.3). This HSI serves as the foundational data structure for subsequent operations. ② The proposed parallel strided sampling (Sec. 3.4.1) is employed, which utilizes the HSI's 0-level sub-block indices to efficiently identify target points. ③ For each target point, neighbor search is then performed within its corresponding adaptive search region (HSI's 1-level indices) using the Lock-free Inverse Neighbor Search mechanism (Sec. 3.4.2). This process identifies both shared neighbors and unique neighbors for each target point, which is organized by the HSI's 2-level neighbor indices. ④ Concurrently with the neighbor search stage, and in line with the compute-once-reuse strategy, the feature update operation (as in Equ. 2) is performed and updated features are stored, ready for reuse. This stage can overlap with neighbor search due to the delayed aggregation. Finally, branch-free segmentation (Sect. 3.4.3) is applied to efficiently combine the pre-computed features of shared neighbors and unique neighbors for each target point.

3.3 Data Orchestration

We transform the complex (sub-blocks, search region, and unique vs. shared neighbor status) relations into the following delegate data layout to avoid the padding and conditional branching issues shown in Fig. 9.

3.3.1 Level 0: Data Layout for Sub-blocks.

Level 0 is designed for sub-block spatial partitioning, which involves organizing points into sub-blocks. The straightforward method allocates fixed memory segments sized for the entire point

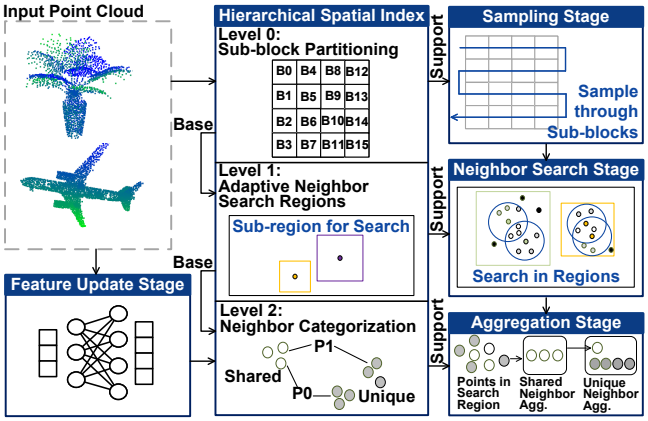

Figure 8. Optimized PCNN architecture and execution pipeline via the proposed PointShuffler engine.

cloud (N points) to each sub-block due to the unknown point count per sub-block prior to memory allocation, resulting in significant memory wastage. In contrast, our spatial coherent organize strategy embeds sub-block spatial distributions directly into memory addresses by storing points from the same sub-block contiguously, thereby efficiently representing spatial relationships using only the original point cloud's memory footprint.

As shown in Fig. 9 (c), the sub-block index comprises two parts: 'point to sub-block' and 'sub-block to point.' The point-to-sub-block part uses a one-dimensional array to record the sub-block ID for each point, enabling the retrieval of a point's corresponding sub-block ID. The sub-block-to-point part reorders point indices contiguously, grouping points from the same sub-block into a single data segment, making block-level point data processing thread-friendly. Additionally, matched length and offset arrays indicate the starting address and data length of each sub-block's data segment, transforming the sub-block's spatial distribution into a deterministic memory layout, thus avoiding worst-case padding.

3.3.2 Level 1: Index of Neighbor Search Regions.

To avoid redundant global searches, we construct local subspaces to reduce the neighbor search region. Level 1 of the HSI defines neighbor search regions based on sub-blocks. Conventionally, neighbor searches for target points are restricted to their respective sub-blocks, resulting in significant neighbor loss for points near sub-block boundaries. Our solution introduces an adaptive hop-based mechanism that dynamically includes h-hop neighboring sub-blocks in the current sub-block's search region:

For each sub-block, we only need to record the IDs of neighboring sub-blocks within an h-hop range to construct the neighbor search index. Combined with the sub-block index, this enables indirect indexing of points in the neighbor search region. As illustrated in Fig. 9 (b) for the 1-hop case, sub-block B9 includes eight additional 1-hop neighboring sub-blocks in its neighbor search region R9, ensuring that even target points near B9's boundary (e.g., point P3) retain access to neighboring points outside their own sub-block without loss.

3.3.3 Level 2: Index of Neighbors.

Level 2 encodes neighbors as shared neighbor, unique neighbor, and non-neighbor for computation reuse. Traditional flag-based methods require annotating all points for all target points, leading to branch divergence during traversal. Our approach transforms neighbor relationships into data ordering, where each target point's memory segment is partitioned into contiguous regions for shared neighbors, unique neighbors, and non-neighbors. This representation eliminates explicit flags by leveraging predictable data boundaries, enabling divergence-free execution on GPUs.

As shown in Fig. 9 (a), the neighbor index includes all points within each target point's neighbor search region. The neighbor indices for all target points are organized in a segmented, contiguous distribution, using associated length and offset arrays to indicate the address of each target point's neighbor index. Similar to the Level 0 sub-block index, this approach avoids worst-case padding issues caused by uneven neighbor distributions. Within each target point's neighbor index segment, shared unique, and non-neighbors are sequentially arranged in sub-segments, with two sets of length data recording the addresses of all the neighbors. This maps the scattered, unordered neighbor relationships into segmented data, enabling threads to process different point types along a known data path without branching.

3.3.4 Efficient Parallel Construction of HSI on GPU.

Algorithm 1 presents the construction details of HSI. ① The Level 0 data layout generation begins by initializing writeOffset and blockLength (lines 1-2). Here, writeOffset represents the intra-segment offset address for each sub-block during the construction of the sorting-based data layout, with all D^3 sub-blocks initialized to zero. The blockLength array records the point count per sub-block and is also initialized to zero. ② Through parallel GPU threads, each point's sub-block index is identified, and the corresponding blockLength counter is atomically updated (lines 3-6). ③ Following this, an exclusive prefix sum operation computes the starting address (blockOffset) for each sub-block in the final data layout (line 7). ④ The final data layout then proceeds in parallel: each thread reads the previous computed mapping relationship from point to sub-block, calculates the target address using blockOffset plus the current writeOffset, performs the data write, and increments the writeOffset atomically (lines 8-12). For Level 1 neighbor search region construction (lines 13-15), ⑤ each sub-block undergoes parallel processing to establish its h-hop sliding window coverage through boundary condition evaluation. The Level 2 neighbor index generation (lines 16-21) involves three coordinated phases: ⑥ First, the sampling stage identifies target points (line 16). ⑦ Next,

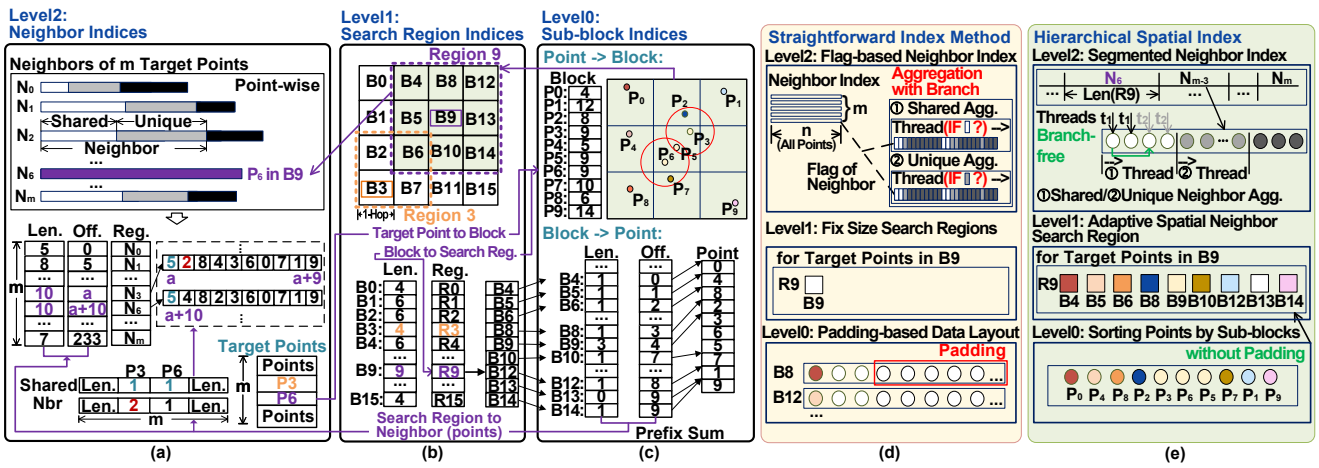

Figure 9. Our proposed Hierarchical Spatial Index (HSI), which efficiently expresses three-level spatial relationships: (a) Level 0: Sub-block Index. (b) Level 1: Neighbor Search Region Index. (c) Level 2: Neighbor Index. (d)-(e): Our HSI is able to avoid padding and enable branch-free processing compared with the straightforward indexing method.

a prefix sum operation calculates per-sub-block neighbor search region sizes, enabling proper memory allocation and initialization for each target point's search space based on its sub-block location. ⑧ Finally, the neighbor search stage employs sorting-based neighbor representation to establish branch-free aggregation.

Algorithm 1 Parallel Construction of HSI

Require: Point Cloud X, Sub-block Number D^3, h-hop Size h
Ensure: Cascaded Spatial Index
◁ *Generating Level 0: Data Layout*
Initialize $writeOffset \leftarrow \{0\}$
Initialize $blockLength \leftarrow \{0\}$
for each x_i in X parallel **do**
  g, $point2Block[x_i] \leftarrow$ GetBlock (x_i, D^3)
  $blockLength[g] \leftarrow blockLength[g]$.AtomicAdd(1)
end for
$blockOffset \leftarrow$ PrefixSum($blockLength$)
for each x_i in X parallel **do**
  $g \leftarrow point2Block[x_i]$
  $writeAddress \leftarrow$ $blockOffset[g] + writeOffset[g]$.AtomicAdd(1)
  $dataLayout \leftarrow$ Update $(x_i, writeAddress)$
end for
◁ *Generating Level 1: Neighbor Search Region Index*
for each g in $blocks$ parallel **do**
  $searchRegion[g] \leftarrow$ H-hopRegion $(g, h, blocks)$
end for
◁ *Generating Level 2: Neighbor Index*
$targetPoint \leftarrow$ Sample(X)
for each x_j in $targetPoint$ parallel **do**
  $g \leftarrow point2Block[x_j]$
  $neighborIndex[x_j]$, $neighborOffset$, $neighborLength$ $\leftarrow$ PrefixSum($blockLength$, $blockOffset$, $searchRegion[g]$)
  // Initialize
  $neighborIndex[x_j] \leftarrow$ NeighborSearch(x_j)
end for

3.4 Operation Parallelization

3.4.1 Parallel Strided Sampling based on HSI.

In Point Cloud Neural Networks (PCNNs), the sampling stage aims to select a representative subset from large-scale input point clouds. Simple random sampling is highly parallelizable, but it is unstable, and often leads to information loss in sparse regions and oversampling in dense ones. Prevalent methods like Farthest Point Sampling (FPS) ensure spatial uniformity and coverage through extensive distance calculations, but their intrinsically sequential, point-by-point selection process fundamentally limits parallel execution.

Key Observation. Our Level 0 data structuring in HSI yields two inherent properties: (1) *3D Spatial Locality Preservation in Memory*: Sub-blocks are indexed according to their XYZ coordinates, and points are subsequently sorted based on their assigned sub-block index. This ensures that points spatially adjacent in 3D are also mapped to nearby locations within the index array. Consequently, applying uniform strided sampling directly to these sorted point indices in memory–without requiring any distance computations–approximates a spatially comprehensive traversal, akin to the coverage achieved by FPS. (2) *Intra-Block Data Locality*: Points within the same 3D sub-block are indexed into contiguous memory locations, which is highly friendly to parallel processing.

Shown in Fig. 10, we propose the Parallel Strided Sampling based on HSI, which directly exploits this structured memory organization, to achieve distance-free, point-level parallel sampling while ensuring spatial coverage, addressing the sequential dependency issue in the sampling stage highlighted in **Challenge 3**. Specifically, given the total number of input points N_{total} and the desired number of sampled points M_{sample} for the current network layer, the sampling process is executed with high parallelism. Specifically, we

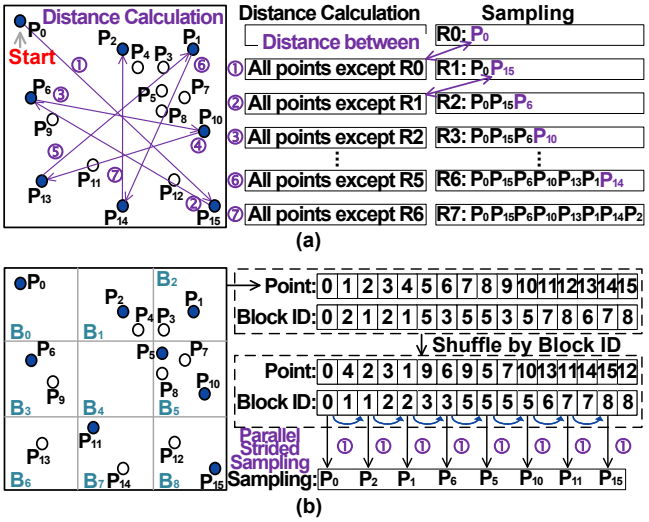

Figure 10. Comparison of (a) farthest point sampling (FPS) and (b) parallel strided sampling (ours).

compute a sampling stride $S = \lfloor N_{total}/M_{sample} \rfloor$. The j-th sampled point (where $0 \le j < M_{sample}$) is then identified as the logical $(j \times S)$-th point within the globally ordered, linearized sequence of all points established by the Level 0 memory structuring. Each GPU thread can independently calculate the memory address of its designated sampled point and retrieve the associated data. This approach achieves a significant reduction in computational complexity from $O(N_{total} \cdot M_{sample})$ for FPS to nearly $O(M_{sample})$, primarily dominated by memory access and index arithmetic.

3.4.2 Lock-free Inverse Neighbor Search.

A significant challenge in leveraging shared neighbors for computation reuse (Sec. 2.3, ②) is the serial bottleneck in identifying these shared points, which requires synchronization after individual neighbor searches complete (**Challenge 3**). PointShuffler addresses this with a novel Lock-free Shared Neighbor Identification mechanism, which can be seamlessly integrated into the neighbor search process with minimal extra overhead.

As shown in Alg. 2, the core insight is an inverse identification strategy. Instead of directly intersecting neighbor sets in a serial synchronized manner, we identify points that are *not* shared neighbors instead. Specifically, for each sub-block b_j, any point x_i within its search region R_j that is *not* a neighbor to any target point x'_k in b_j cannot be a shared neighbor for b_j.

To achieve this, we use a boolean mask, `nonNeighborMask[`b_j`, ·]`, initialized to `False` for each sub-block b_j and its candidate points. When a thread processing x'_k identifies such a non-neighbor x_i (i.e., $x_i \in R_j \backslash N_k$), it directly sets `nonNeighborMask[`$b_j$`, `$x_i$`] = True` (Alg. 2, line 13). Crucially, writing `True` to the `nonNeighborMask` is lock-free, as the correctness of marking x_i as 'not a shared neighbor' is guaranteed if *any single thread* succeeds; this idempotent operation naturally handles concurrent writes without requiring locks. Finally, after all target points in b_j have been processed, any point x_s for which `nonNeighborMask[`b_j`, `x_s`]` remains `False` is confirmed as a shared neighbor for that sub-block.

Algorithm 2 Lock-free Inverse Neighbor Search

Require: Target Points X_j, Neighbor Search Region Index *searchRegion*, Sub-block Index *subBlock*
Ensure: Shared Neighbor Set *sharedNeighbor*
1: Initialize $nonNegihborMask[D^3, n] = False$ //For All D^3 Sub-blocks, and n Points
2: **for** each x'_j in X' parallel **do**
3: $b_j \leftarrow subBlock[x'_j]$
4: $R_j \leftarrow searchRegion[b_j]$
5: **for** each *block* in R_j **do**
6: **for** each x_i in *block* parallel **do**
7: $d_{ji} \leftarrow \text{DISTANCE}(x'_j, x_i)$
8: **end for**
9: $N_j \leftarrow \text{BALLQUERY}(d_j)$ *or* $\text{KNN}(d_j)$
10: $nonNeighbor_j \leftarrow R_j \setminus N_j$
11: **for** each x_i in $nonNeighbor_j$ parallel **do**
12: $nonNeighborMask[b_j, x_i] \leftarrow True$ // Lock-free
13: **end for**
14: **end for**
15: **end for**
16: $sharedNeighbor \leftarrow \text{INVERSE}(nonNeighborMask)$

Algorithm 3 Branch-free Segmented Aggregation Pipeline.

Require: Neighbor Index N, Updated Feature $\tilde{X}$, Sub-block Index *subBlock*, Target Points $X' \in R^{n \times F'}$, Number of Feature Channels F'
Ensure: OutPut PointCloud $\hat{X}'$
◁ ***Block-level*** *Shared Neighbor Aggregation*
1: **for** b in subBlock parallel **do**
2: $x_b \leftarrow choose1Point(b)$
3: $sharedSeg, uniqueSeg \leftarrow N(x_b)$
4: $length, offset \leftarrow sharedSeg$
5: **for** f in F' parallel **do**
6: **for** i in $range(length)$ **do**
7: $R_b^f \leftarrow Aggregation(\tilde{X}^f_{offset+i})$ //Reuse Updated Features
8: **end for**
9: **end for**
10: **end for**
◁ ***Target-level*** *Unique Neighbor Aggregation*
11: **for** x'_j in X' parallel **do**
12: $sharedSeg, uniqueSeg \leftarrow N(x'_j)$
13: $length, offset \leftarrow uniqueSeg$
14: $b_j \leftarrow subBlock(x'_j)$
15: **for** f in F' parallel **do**
16: $\hat{X}'^f_j \leftarrow R^f_{b_j}$ //Reuse Shared Aggregation
17: **for** i in $range(length)$ **do**
18: $\hat{X}'^f_j \leftarrow Aggregation(\tilde{X}^f_{offset+i})$ //Reuse Updated Features
19: **end for**
20: **end for**
21: **end for**

3.4.3 Branch-free Segmented Aggregation for Computation Reuse.

Our Branch-free Segmented Aggregation

(Alg. 3) achieves efficient, SIMT-friendly processing based on the proposed Hierarchical Spatial Index (HSI). As HSI (level-2) organizes neighbor indices into distinct, contiguous memory segments shown in Fig. 9 (shared neighbor segment followed by unique neighbor segment), aggregation within these segments with warps becomes inherently branch-free. The pipeline has two main stages:

The first stage, block-level shared neighbor aggregation (Lines 1-10 in Alg. 3), is designed to pre-compute aggregated features from neighbors within the same spatial sub-block. This stage executes in parallel for each sub-block b (Line 1). Within each sub-block, a single random point x_b is chosen (Line 2), and its shared neighbor segment (denoted as `sharedSeg` containing an `offset` and `length`) is identified using the HSI from the neighbor index $N(x_b)$ (Line 4). The core aggregation for these shared neighbors is then performed. To ensure workload balance across GPU threads, especially given the potentially uneven distribution of neighbors, the aggregation process is parallelized across all feature channels F' (Line 5). For every feature channel f and each neighbor index k = `offset` + i within the `sharedSeg` (Lines 6-7), the corresponding updated feature $\tilde{X}_k^f$ is aggregated into an intermediate, block-level result R_b^f. This step also reuses the updated features $\tilde{X}$ in a delayed aggregation manner that avoids repeated computations for shared neighbors.

The second stage, target-level unique neighbor aggregation (Lines 11-21 in Alg. 3), finalizes the feature aggregation for each individual target point $x'_j \in X'$. This stage also operates in parallel for all target points (Line 11). For a given target point x'_j, its associated sub-block b_j is first determined (Line 14). The output feature $\hat{X'}_j^f$ for each channel f is then initialized by directly reusing the pre-computed shared aggregation result $R_{b_j}^f$ from the corresponding sub-block b_j (Line 16). Subsequently, the algorithm retrieves the unique neighbor segment (`uniqueSeg`) for x'_j via HSI from $N(x'_j)$ (Lines 12-13). The features of these unique neighbors are then aggregated into $\hat{X'}_j^f$ (Line 18), again reusing the updated features $\tilde{X}_k^f$ (where k = `offset` + i). This two-stage process ensures that shared computations are performed only once per block and reused, while unique computations are handled efficiently per target point, both in a branch-free execution way for high GPU utilization.

3.5 Memory Model

Tab. 1 lists the memory overhead formulas for the key components. For PointShuffler, the index metadata grows $O(n)$ with the number of points n, providing linear scalability. Specifically, the sub-block indices consist of the length and offset arrays (Size: $2D^3$) for sub-blocks, the point-to-sub-block index array (n), and the re-sorted point indices (n) (Fig. 9(c)). The search region indices comprise the length and offset arrays ($2D^3$) for recording the neighbor search regions of each sub-block, along with the array of sub-block indices ($[(1+h)^3+1]D^3$) contained within those regions (Fig. 9(b)). The neighbor indices include arrays recording the lengths of neighbors and shared neighbors for each target point ($2m$), as well as the point indices within the neighbor search regions and their corresponding length and offset arrays ($(n+1)D^3$) (Fig. 9(a)). The neighbor search features represent distance information used for neighbor search, where O_{dis} is a prior statistic denoting the total number of points encompassed across all target points' neighbor search region, owing to the adaptive region effectively reducing the search region size, O_{dis} is far smaller than the vanilla baseline's $2mn$. The update features contain the output features from the MLP computations. Finally, the intermediate result corresponds to the features reused during aggregation.

Table 1. Memory model of PointShuffler. n is the total point cloud count, m is the target points, D^3 is the sub-blocks count, C_{in} is the input feature channel count, C_j denotes the output channel count of the jth MLPs layer in feature update, k is the top-k value in the neighbor search, and h denotes the hop number in the h-hop searching region.

Parts	Vanilla PCNN w/ delayed aggregation	PointShuffler (ours)
Sub-block Indices	\	$(2n + 2D^3) \times Sizeof(int)$
Search Region Indices	\	$[(1+2h)^3+3]D^3 \times Sizeof(int)$
Neighbor Indices	$mk \times Sizeof(int)$	$(n+1)D^3 \times Sizeof(bool)$ $+2m \times Sizeof(int)$
Neighbor Search Features	$2mn \times Sizeof(int)$	$2O_{dis} \times Sizeof(int)$
Update Features	$n(C_1 + C_2 + ... + C_j)$ $\times Sizeof(float)$	$n(C_1 + C_2 + ... + C_j)$ $\times Sizeof(float)$
Interm. Result	\	$D^3 C_j \times Sizeof(float)$
Other	$n(C_{in}+3) + m(C_j+3) + (C_{in}C_1 + C_1C_2 + ...C_{j-1}C_j)$	

4 Performance Evaluation

4.1 Experimental Setup

4.1.1 Evaluated PCNNs and Datasets. We evaluate our engine and all baselines on four PCNN models and four well-known datasets with two mainstream tasks, as listed in Table 2.

Table 2. PCNN models and datasets.

Dataset	Point Cloud Network	Application Domains
ModelNet40 [36]	PointNet++(c) [25] DGCNN(c) [35] PointNeXt [26] PointVector [6]	Classification
ShapeNet [2]	PointNet++	Segmentation
S3DIS[1]	PointNet++	Segmentation
KITTI[11]	PointNet++	Detection

4.1.2 Compared Baselines. We accelerate the PCNN models with each baseline's GPU acceleration method and compare their performance in terms of task accuracy, end-to-end

latency, and GPU memory consumption. The *Vanilla* baseline denotes the native GPU implementation of the models (no redundancy-removal optimizations). To ensure fair comparisons between PointShuffler (GPU-based) with four leading PCNN ASIC accelerators, Mesorasi [9], ParallelNN [4], GDPCA [3], and SimDiff [17], we evaluate Mesorasi using its official GPU implementation (denoted *Mesorasi-G*). For the latter three, we faithfully re-implement their redundancy optimization techniques on the GPU platform, and get their GPU-adapted counterparts *ParallelNN-G*, *GDPCA-G*, and *SimDiff-G*. Details are shown in Tab. 3.

Table 3. The baselines used in this paper (✔ and ✖ denote w/ and w/o redundancy optimization)

Key Operations	Vanilla Baseline	Mesorasi-G	ParallelNN-G	GDPCA-G	SimDiff-G
Partitioning	✖	✖	✔	✔	✔
Sampling	✖	✖	✖	✖	✔
Neighbor Search	✖	✖	✔	✔	✔
Feature Update	✖	✔	✔	✔	✔
Aggregation	✖	✖	✖	✔	✔

4.1.3 Hardware platforms.

Our target is on-device edge deployment. We therefore profile single-sample inference (batch size = 1) on three representative NVIDIA GPUs: Jetson TX2 (Pascal-class embedded SoC) for resource-constrained AR/VR devices and RTX 3080 (Ampere) for typical automotive platforms. We also report A100 results as a high-end scaling platform.

4.1.4 Hyperparameter Exploration.

Because two hyperparameters, D and h, affect both PointShuffler's latency and accuracy, we first quantified their effects and then fixed the configuration used in all subsequent experiments. D means the space is partitioned into D^3 sub-blocks, and h determines the extent of the neighbor search region. Fig. 11 reports the accuracy exploration results, compared with full-space kNN, h=2 yields nearly unchanged accuracy, h=1 causes a slight drop, and h=0, which equals to the fix-size partition, leads to a substantial decline because the search is confined to the target sub-block and misses neighbors across boundaries. In the latency aspect, increasing D creates more sub-blocks to index and aggregate, raising partitioning and block-level shared-aggregation costs. In contrast, decreasing D or increasing h enlarges each search region, increasing candidate neighbors and thus the time for neighbor search. As shown in Fig. 12, these effects trade off to yield a latency minimum at $D = 10$ and $h = 1$, which also satisfies the accuracy targets. So we set it as the default for all the following experiments.

4.2 Experimental Result

4.2.1 Accuracy.

We report the task accuracy of the baselines on ModelNet40 (classification) and ShapeNet (segmentation) in Fig. 13. Our method attains accuracy comparable

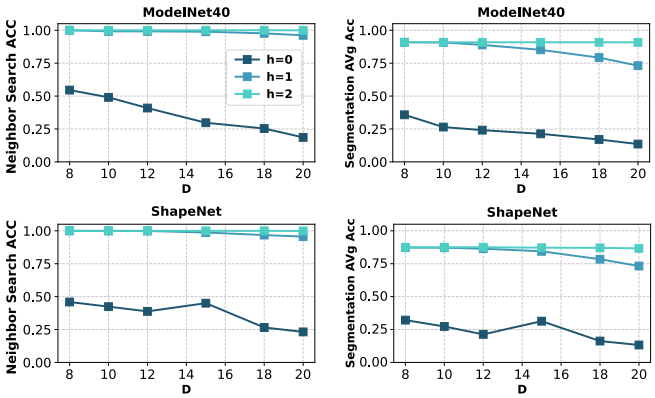

Figure 11. Neighbor search and task accuracy of adaptive searching region under different D and h based on the PointNet++ model and different datasets (i.e., ModelNet40 and ShapeNet).

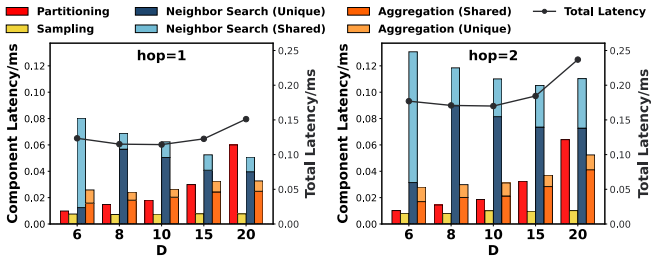

Figure 12. The impact of hyperparameters D and h-hop on execution latency based on the PointNet++ model and the ModelNet40 dataset.

to the vanilla baseline (which achieves the best accuracy), indicating that all our optimizations do not degrade accuracy. Similarly, for Mesorasi-G, GDPCA-G, and SimDiff-G, their redundant elimination methods do not introduce noticeable accuracy degradation. ParallelNN-G shows a slight drop, likely because its octree-based neighbor search yields incomplete neighborhoods compared with exact KNN.

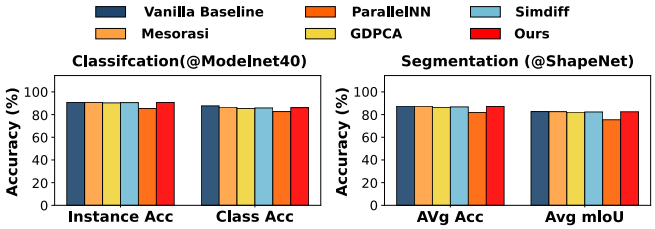

Figure 13. Accuracy comparison of our engine and all the baselines, based on the PointNet++ model and two datasets for different downstream tasks.

4.2.2 End-to-End latencies.

Shown in Fig. 14, on average, our method achieves 6.15× speedup compared with

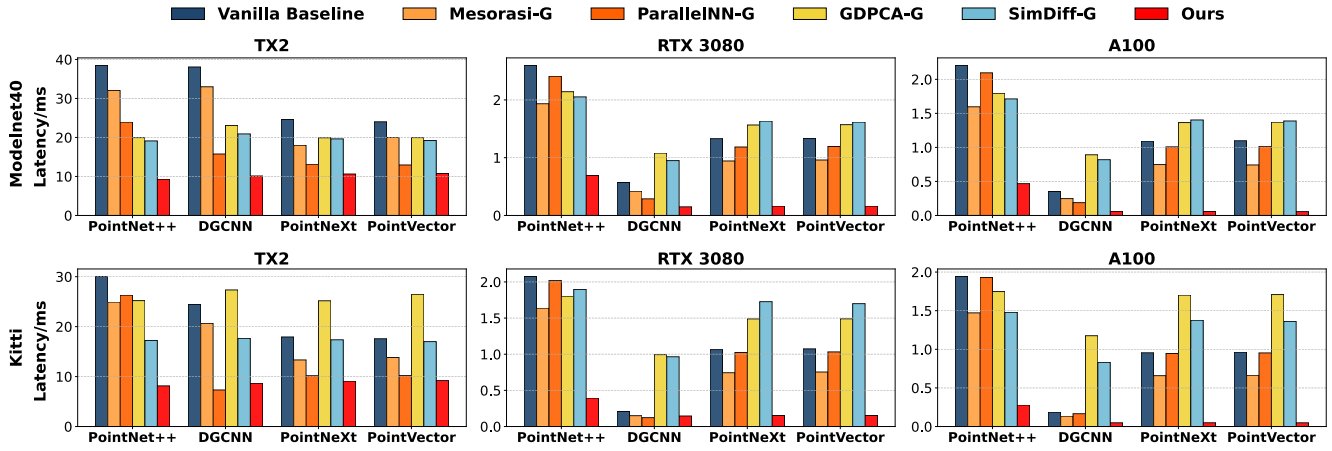

Figure 14. The end-to-end latency evaluation on our engine and all the baselines among different PCNN models, datasets, and GPU platforms.

Vanilla Baseline, Mesorasi-G, ParallelNN-G, GDPCA-G, and SimDiff-G. Mesorasi is 1.34× faster than the vanilla GPU by removing update redundancy with full parallelism; however, in models where the update stage constitutes only a small fraction of the pipeline (e.g., DGCNN, PointNeXt, PointVector), its benefit is limited. ParallelNN accelerates neighbor search via an octree-based local search, but octree construction, updating, and pointer-heavy traversal are inefficient on GPUs, so its speedup is far below its ASIC design. GDPCA and SimDiff attempt to eliminate redundancy in neighbor search and aggregation, yet they introduce control-flow divergence, irregular memory access, and serial dependencies that are SIMT-unfriendly; for models with few updates, these effects can even yield negative gains compared with vanilla. In contrast, our approach removes redundancy in a SIMT-friendly way, resulting in the best end-to-end latency across all settings.

We also observe performance variations of PointShuffler across different hardware platforms, datasets, and PCNN models. Across GPUs, PointShuffler achieves greater acceleration on high-end GPUs (e.g., A100, RTX 3080) than TX2. On the one hand, for compute-intensive stages such as neighbor search and sampling, our method increases parallelism. As a result, it improves SM and bandwidth utilization on high-end GPUs, leading to larger speedups. As shown in Fig. 18, for the sampling stage, the speedup is 114.08× on RTX 3080 versus 58.18× on TX2. Similarly, for the neighbor search stage, the speedup reaches 4.72× on RTX 3080 compared to 2.25× on TX2. On the other hand, for the aggregation stage, where control overhead is more significant, older GPU architectures like the Pascal-based TX2 benefit more from our method's reduction in control-flow flags, due to their less sophisticated warp schedulers, and achieve higher speedup. According to Fig. 18, the aggregation stage achieves a speedup of 9.90x on RTX 3080 and 14.96x on TX2. Overall, since neighbor search and sampling constitute the majority of the processing time, PointShuffler delivers better overall performance on higher-end GPUs.

Across datasets, PointShuffler's speedup differs since different spatial distributions of points lead to different levels of neighbor overlap and reuse. KITTI, as a real-world dataset, is typically very dense in the central region, whereas ModelNet40, an object-level dataset, is relatively even overall. For example, the reuse rates of shared neighbors for the two datasets are 75.21% and 43.39%, respectively, and the aggregation stage achieves average speedups of 3.46× on KITTI and 1.98× on ModelNet40 (Fig. 14).

Across PCNN models, per-stage workloads differ, resulting in different latency breakdowns and overall speedups. For instance, the number of downsampled points affects the workload of the sampling and neighbor search stages, while the number of feature channels influences the workload of the feature update and aggregation stages. Taking the ModelNet40 in Fig. 14 as an example, the sampling stage accounts for 24.98% of the total latency in PointNet++ but 0.00% in DGCNN, whereas the neighbor search stage constitutes 28.03% in PointNet++ compared to 85.81% in DGCNN.

4.2.3 Memory Consumption. Figure 15 shows average per-cloud memory divided into indices, features, and others: indices include necessary spatial-partition metadata such as HSI; features include all per-stage feature tensors and intermediate results; others are model weights and the input/output clouds.

Overall, our method's memory consumption is 3.86 MB and reduces total memory by 28.28–81.14% relative to the baselines. For indexing, compared with the vanilla baseline with almost no indexing, HSI adds only a small overhead (1.11 MB). Compared with GDPCA-G and SimDiff-G, our HSI uses light metadata to avoid heavy worst-case padding and yields large savings of 3.90 MB. For features/intermediates,

HSI, together with the adaptive search region, reduces the sampling and neighbor search intermediates of candidate points by localizing the sampling/search region. Compared with vanilla, our shared neighbor caching introduces a small extra buffer (0.49 MB) for update/aggregation reuse but removes a large amount of duplicate storage (15.19 MB).

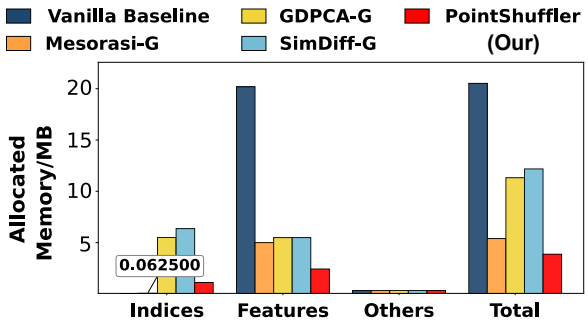

Figure 15. Memory consumption evaluation based on the PointNet++ model and the ModelNet40 dataset.

4.2.4 Scalability with Input Point Count.

Although deployed PCNNs typically process no more than ~4k points, we stress-tested scalability by growing the point set from 1k to 65k. Figure 16 shows that the vanilla GPU pipeline's latency rises sharply (from 5.98 ms at 2k points to 1.75 s at 65k), whereas PointShuffler remains below ~2.1 ms across the range, yielding speedups that increase from about 10× to 850×. Memory follows the same pattern: the vanilla implementation reaches ~17 GB at 65k points, while PointShuffler stays below ~4 GB thanks to locality-preserving indexing and reuse. Our PointShuffler has much flatter growth with input size.

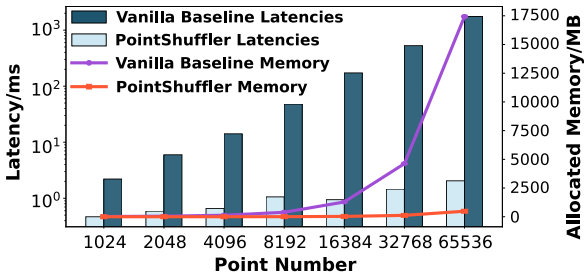

Figure 16. Scalability with the increasing number of input points based on the PointNet++ model and the ModelNet40 data set.

4.3 Ablation Study

4.3.1 Effect of Network Architecture Transformation.

As the transformed PCNN cannot be executed directly on GPUs, we study its effectiveness through evaluating its theoretical GPU cost under an ideal SIMT model, without index construction, padding, or branch divergence overheads. As shown in Fig. 17, compared with the delayed-aggregation baseline that already reduces updates, the proposed adaptive search region and shared-neighbor cache effectively reduce sampling by 99.12%/98.81%, neighbor search by 78.82%/55.57%, and aggregation by 89.90%/93.32% on ModelNet and KITTI, respectively, with average memory reduced by 28.28% in theory. The gains are larger on KITTI because its denser scenes enable higher reuse. Moreover, compared with the measured runtime of the fully optimized PointShuffler (all optimizations enabled, our implementation closely approaches the theoretical limit.

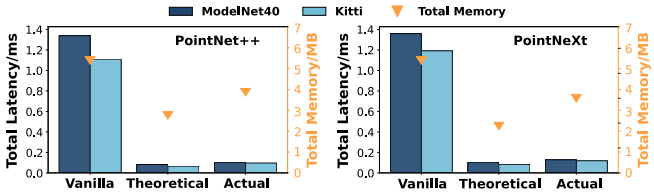

Figure 17. Effect of the network architecture transformation on theoretical GPU latency. *Theoretical* denotes the network's theoretical GPU cost after the network architecture transformation, whereas *Actual* denotes the fully optimized PointShuffler.

4.3.2 Effect of Data Orchestration.

We design two variants to assess the effect of the proposed HSI: the mark × denotes a naive feature-reuse implementation that uses a padding-based index for sub-block indexing and randomized indices for the search region and neighbor status; the mark "cpu" denotes the variant constructing HSI on the CPU (similar to the idea of PagedAttention for K/V cache indices). Results show that the naive variant causes a 253.46× increase in memory due to padding; moreover, without HSI, LNS and BSA suffer from irregular control flow and memory access, performing an average 18.11× worse than vanilla aggregation without reuse. PSS maintains an unchanged speed when using a padding-based index. For the CPU variant, CPU-side HSI construction adds an average of 20.33% overhead to end-to-end time, demonstrating the effectiveness of our parallel on-GPU construction.

4.3.3 Effect of Operation Parallelization.

We construct ablation variants to study the effectiveness of the three proposed optimizations: PSS (parallel strided sampling; × denotes replacement by FPS), LNS (lock-free inverse neighbor search; × denotes KNN), and BSA (branch-free segmented aggregation; × denotes vanilla aggregation without reuse). Because these parallel optimizations are tightly coupled with the transformation and indexing optimizations, each variant is evaluated with enabling the minimal subset of *Network Architecture Transformation* and *Data Orchestration* necessary to maximize their effectiveness. As shown in Fig. 18, all three

techniques substantially reduce the latency of their respective stages (PSS for sampling, LNS for neighbor search, BSA for aggregation), removing the vast majority of per-stage cost and demonstrating that we effectively address reuse-induced control flow and memory irregularities, as well as the challenges from sequential dependencies.

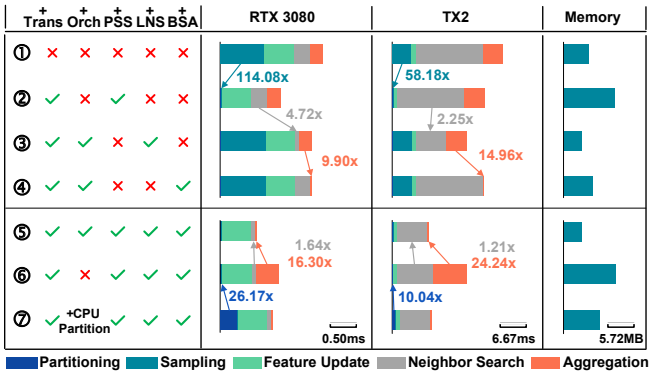

Figure 18. Effect of all the proposed optimizations on latency and memory measured on PointNet++ and ModelNet40.

5 Related Work

5.1 System-Level Acceleration for Point Cloud Deep learning

For 3D rendering tasks, CICERO [8] accelerates neural rendering on GPUs by sparse radiance warping and channel-major memory access to eliminate irregular DRAM traffic and SRAM bank conflicts. 3DGS [15] accelerates radiance-field rendering on GPUs with tile-based Gaussian-splat rasterization that applies global radix sorting and per-tile shared-memory blending. For 3D perception tasks, voxel-based acceleration methods on GPU have been extensively studied through sparse convolution (spConv) optimization. Torchsparse/++ [30, 31] accelerate spConv by adaptive GEMM grouping and optimizing data movement with fused, locality-aware accesses. Minuet [40] introduces sort-based kernel-map construction and autotuned GMaS tiling with padding-efficient GEMM grouping for spConv. In contrast, point-based models directly process raw points, preserving fine-grained geometric structures to avoid information loss caused by gridding, but irregular point operations and computational redundancy typically make them slower than voxel-based counterparts. Prior efforts to overcome these PCNN bottlenecks are predominantly ASIC-oriented [3, 9, 17]. To our knowledge, PointShuffler is the first GPU-based acceleration approach for general point-based PCNNs.

5.2 Redundancy Elimination and Reuse Techniques for Neural Networks

Researchers have employed both redundancy elimination and reuse techniques to improve runtime efficiency across diverse neural-network tasks. For redundancy elimination, BlockDance [42] and PvNeXt [32] reduce spatiotemporal redundancy in point-cloud video. However, these methods are limited to the algorithmic improvements. TGOpt [34] mitigates intra-batch and temporal redundancies in temporal GNN inference via deduplication of repeated node–timestamp pairs and precomputation of frequent time encodings. FlashSparse [28] reduces zero-heavy block-construction redundancy in SpMM via a swap-and-transpose MMA scheme on Tensor Cores, and validates it in GNN. Nevertheless, they fail to address the complex redundancy inherent in PCNNs.

Like our reuse of intermediates, prior work has also explored reuse-based accelerations in 2D vision [7, 12, 14, 39] (e.g., cross-frame similarity, feature map caching) and 3D rendering [5, 13, 21, 27] (e.g., spatial indexing structures, pre-baked radiance). In most of these works, reuse strategies rely on inherently structured, grid-aligned data and indices (e.g., pixel/voxel/raster grids). Point clouds, however, are irregular and unstructured; thus, these reuse techniques are not applicable to point clouds. Similar reuse also appears in serving systems. KV cache [10, 16, 24, 38, 41, 44] enables reuse of past keys/values across auto-regressive step in long-lived, concurrent LLM serving. In retrieval and vector search systems, researchers improve performance by constructing and reusing indices optimized for repeated queries [19, 20, 22, 33]. However, these methods are designed for long-term, frequent-access scenarios, which do not align with the short-term, low-latency index construction and reuse requirements of PCNNs.

6 Conclusion

This work demonstrates that the principal redundancies in PCNNs can be eliminated in a GPU-compatible manner by jointly reformulating parallelizing core computations and transforming irregular memory access into predictable patterns suited to SIMT execution, thereby preserving accuracy while enabling commodity GPUs for latency-sensitive 3D perception. Future work will extend these redundancy-elimination principles to point-cloud video streams by leveraging inter-frame structure for streaming index and feature reuse, and to transformer-based point-cloud models by structuring tokens and attention caches for efficient GPU execution.

Acknowledgments

This paper is supported by the National Natural Science Foundation of China (Grant 62302529, Grant 62572177, Grant 62427808, and Grant 62472181), and the Science and Technology Innovation Program of Hunan Province (Grant 2025RC3079).

References

[1] Iro Armeni, Ozan Sener, Amir R Zamir, Helen Jiang, Ioannis Brilakis, Martin Fischer, and Silvio Savarese. 2016. 3d semantic parsing of large-scale indoor spaces. In *IEEE Conference on Computer Vision and Pattern*

Recognition (CVPR). 1534–1543.

[2] Angel X. Chang, Thomas Funkhouser, Leonidas Guibas, Pat Hanrahan, Qixing Huang, Zimo Li, Silvio Savarese, Manolis Savva, Shuran Song, Hao Su, Jianxiong Xiao, Li Yi, and Fisher Yu. 2015. *ShapeNet: An Information-Rich 3D Model Repository*. Technical Report arXiv.

[3] Cen Chen, Xiaofeng Zou, Hongen Shao, Yangfan Li, and Kenli Li. 2023. Point cloud acceleration by exploiting geometric similarity. In *IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 1135–1147.

[4] Faquan Chen, Rendong Ying, Jianwei Xue, Fei Wen, and Peilin Liu. 2023. Parallelnn: A parallel octree-based nearest neighbor search accelerator for 3d point clouds. In *IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 403–414.

[5] Zhiqin Chen, Thomas Funkhouser, Peter Hedman, and Andrea Tagliasacchi. 2023. Mobilenerf: Exploiting the polygon rasterization pipeline for efficient neural field rendering on mobile architectures. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 16569–16578.

[6] Xin Deng, WenYu Zhang, Qing Ding, and XinMing Zhang. 2023. Pointvector: A vector representation in point cloud analysis. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 9455–9465.

[7] Utsav Drolia, Katherine Guo, Jiaqi Tan, Rajeev Gandhi, and Priya Narasimhan. 2017. Cachier: Edge-caching for recognition applications. In *2017 IEEE 37th international conference on distributed computing systems (ICDCS)*. IEEE, 276–286.

[8] Yu Feng, Zihan Liu, Jingwen Leng, Minyi Guo, and Yuhao Zhu. 2024. Cicero: Addressing algorithmic and architectural bottlenecks in neural rendering by radiance warping and memory optimizations. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 1293–1308.

[9] Yu Feng, Boyuan Tian, Tiancheng Xu, Paul Whatmough, and Yuhao Zhu. 2020. Mesorasi: Architecture support for point cloud analytics via delayed-aggregation. In *IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 1037–1050.

[10] Suyu Ge, Yunan Zhang, Liyuan Liu, Minjia Zhang, Jiawei Han, and Jianfeng Gao. 2023. Model tells you what to discard: Adaptive kv cache compression for llms. *arXiv preprint arXiv:2310.01801* (2023).

[11] Andreas Geiger, Philip Lenz, and Raquel Urtasun. 2012. Are we ready for autonomous driving? the kitti vision benchmark suite. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 3354–3361.

[12] Peizhen Guo and Wenjun Hu. 2018. Potluck: Cross-application approximate deduplication for computation-intensive mobile applications. In *Proceedings of the Twenty-Third International Conference on Architectural Support for Programming Languages and Operating Systems*. 271–284.

[13] Peter Hedman, Pratul P Srinivasan, Ben Mildenhall, Jonathan T Barron, and Paul Debevec. 2021. Baking neural radiance fields for real-time view synthesis. In *Proceedings of the IEEE/CVF international conference on computer vision*. 5875–5884.

[14] Loc N Huynh, Youngki Lee, and Rajesh Krishna Balan. 2017. Deepmon: Mobile gpu-based deep learning framework for continuous vision applications. In *Proceedings of the 15th Annual International Conference on Mobile Systems, Applications, and Services*. 82–95.

[15] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 2023. 3D Gaussian splatting for real-time radiance field rendering. *ACM Trans. Graph.* 42, 4 (2023), 139–1.

[16] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*. 611–626.

[17] Yangfan Li, Mengquan Li, Cen Chen, Xiaofeng Zou, Hongen Shao, Fengxiao Tang, and Kenli Li. 2024. SimDiff: Point Cloud Acceleration by Utilizing Spatial Similarity and Differential Execution. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems (IEEE TCAD)* (2024).

[18] Zhe Li, Zhangyang Gao, Cheng Tan, Bocheng Ren, Laurence T Yang, and Stan Z Li. 2024. General Point Model Pretraining with Autoencoding and Autoregressive. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 20954–20964.

[19] Zihan Liu, Wentao Ni, Jingwen Leng, Yu Feng, Cong Guo, Quan Chen, Chao Li, Minyi Guo, and Yuhao Zhu. 2024. Juno: Optimizing high-dimensional approximate nearest neighbour search with sparsity-aware algorithm and ray-tracing core mapping. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 549–565.

[20] Yu A Malkov and Dmitry A Yashunin. 2018. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE transactions on pattern analysis and machine intelligence* 42, 4 (2018), 824–836.

[21] Thomas Müller, Alex Evans, Christoph Schied, and Alexander Keller. 2022. Instant neural graphics primitives with a multiresolution hash encoding. *ACM transactions on graphics (TOG)* 41, 4 (2022), 1–15.

[22] Hiroyuki Ootomo, Akira Naruse, Corey Nolet, Ray Wang, Tamas Feher, and Yong Wang. 2024. Cagra: Highly parallel graph construction and approximate nearest neighbor search for gpus. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 4236–4247.

[23] Chunghyun Park, Yoonwoo Jeong, Minsu Cho, and Jaesik Park. 2022. Fast point transformer. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 16949–16958.

[24] Ramya Prabhu, Ajay Nayak, Jayashree Mohan, Ramachandran Ramjee, and Ashish Panwar. 2025. vattention: Dynamic memory management for serving llms without pagedattention. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. 1133–1150.

[25] Charles Ruizhongtai Qi, Li Yi, Hao Su, and Leonidas J Guibas. 2017. Pointnet++: Deep hierarchical feature learning on point sets in a metric space. *Advances in Neural Information Processing Systems* 30 (2017).

[26] Guocheng Qian, Yuchen Li, Houwen Peng, Jinjie Mai, Hasan Hammoud, Mohamed Elhoseiny, and Bernard Ghanem. 2022. Pointnext: Revisiting pointnet++ with improved training and scaling strategies. *Advances in Neural Information Processing Systems* 35 (2022), 23192–23204.

[27] Christian Reiser, Rick Szeliski, Dor Verbin, Pratul Srinivasan, Ben Mildenhall, Andreas Geiger, Jon Barron, and Peter Hedman. 2023. Merf: Memory-efficient radiance fields for real-time view synthesis in unbounded scenes. *ACM Transactions on Graphics (ToG)* 42, 4 (2023), 1–12.

[28] Jinliang Shi, Shigang Li, Youxuan Xu, Rongtian Fu, Xueying Wang, and Tong Wu. 2025. Flashsparse: Minimizing computation redundancy for fast sparse matrix multiplications on tensor cores. In *Proceedings of the 30th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*. 312–325.

[29] Jingtao Sun, Yaonan Wang, Mingtao Feng, Yulan Guo, Ajmal Mian, and Mike Zheng Shou. 2024. L4D-Track: Language-to-4D Modeling Towards 6-DoF Tracking and Shape Reconstruction in 3D Point Cloud Stream. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 21146–21156.

[30] Haotian Tang, Zhijian Liu, Xiuyu Li, Yujun Lin, and Song Han. 2022. Torchsparse: Efficient point cloud inference engine. *Machine Learning and Systems* 4 (2022), 302–315.

[31] Haotian Tang, Shang Yang, Zhijian Liu, Ke Hong, Zhongming Yu, Xiuyu Li, Guohao Dai, Yu Wang, and Song Han. 2023. Torchsparse++: Efficient point cloud engine. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 202–209.

[32] Jie Wang, Tingfa Xu, Lihe Ding, Xinjie Zhang, Long Bai, and Jianan Li. 2025. PvNeXt: Rethinking Network Design and Temporal Motion for Point Cloud Video Recognition. In *The Thirteenth International Conference on Learning Representations*. https://openreview.net/forum?id=ZsU52Zkzjr

[33] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, et al. 2021. Milvus: A purpose-built vector data management system. In *Proceedings of the 2021 international conference on management of data*. 2614–2627.

[34] Yufeng Wang and Charith Mendis. 2023. Tgopt: Redundancy-aware optimizations for temporal graph attention networks. In *Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*. 354–368.

[35] Yue Wang, Yongbin Sun, Ziwei Liu, Sanjay E Sarma, Michael M Bronstein, and Justin M Solomon. 2019. Dynamic graph cnn for learning on point clouds. *ACM Transactions on Graphics (TOG)* 38, 5 (2019), 1–12.

[36] Zhirong Wu, Shuran Song, Aditya Khosla, Fisher Yu, Linguang Zhang, Xiaoou Tang, and Jianxiong Xiao. 2015. 3d shapenets: A deep representation for volumetric shapes. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 1912–1920.

[37] Yan Xia, Letian Shi, Zifeng Ding, Joao F Henriques, and Daniel Cremers. 2024. Text2loc: 3d point cloud localization from natural language. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 14958–14967.

[38] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2023. Efficient streaming language models with attention sinks. *arXiv preprint arXiv:2309.17453* (2023).

[39] Mengwei Xu, Mengze Zhu, Yunxin Liu, Felix Xiaozhu Lin, and Xuanzhe Liu. 2018. Deepcache: Principled cache for mobile deep vision. In *Proceedings of the 24th annual international conference on mobile computing and networking*. 129–144.

[40] Jiacheng Yang, Christina Giannoula, Jun Wu, Mostafa Elhoushi, James Gleeson, and Gennady Pekhimenko. 2024. Minuet: Accelerating 3D sparse convolutions on GPUs. In *European Conference on Computer Systems*. 786–802.

[41] Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, and Luis Ceze. 2025. FlashInfer: Efficient and Customizable Attention Engine for LLM Inference Serving. In *Eighth Conference on Machine Learning and Systems*. https://openreview.net/forum?id=RXPofAsL8F

[42] Hui Zhang, Tingwei Gao, Jie Shao, and Zuxuan Wu. 2025. Blockdance: Reuse structurally similar spatio-temporal features to accelerate diffusion transformers. In *Proceedings of the Computer Vision and Pattern Recognition Conference*. 12891–12900.

[43] Jie-Fang Zhang and Zhengya Zhang. 2021. Point-x: A spatial-locality-aware architecture for energy-efficient graph-based point-cloud deep learning. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*. 1078–1090.

[44] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, et al. 2023. H2o: Heavy-hitter oracle for efficient generative inference of large language models. *Advances in Neural Information Processing Systems* 36 (2023), 34661–34710.